

Interview Question:

A Synapse dedicated SQL pool query suddenly becomes slow after a large load.

My Answer:

I'd check statistics first because the optimizer relies on them for cardinality estimates. I'd compare when the relevant statistics were last updated with the amount and distribution of new data. I'd also check the execution plan and columnstore health if the problem persists. After updating the necessary statistics, I'd rerun the same query and compare duration and plan behavior.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A large fact table is distributed round-robin and joins are expensive.

My Answer:

I'd inspect where data movement occurs in the execution plan and whether a better distribution key matches the major join pattern. I'd consider hash distribution when there is a stable, high-quality join key and the data can be distributed evenly. I'd also account for skew because a poor distribution key can make one node much heavier. I'd test with representative data before changing the production table.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse table has severe data skew.

My Answer:

I'd identify the distribution column and compare row counts across distributions. If a few key values dominate, hash distribution can concentrate work on a small number of distributions. I'd evaluate another distribution key or a different distribution strategy based on the workload. I'd validate both skew and join movement after the change because eliminating skew is not useful if it creates excessive data movement.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A dedicated SQL pool query has a large data movement step.

My Answer:

I'd inspect the join pattern and table distributions first. Data movement can be necessary, but repeated large movements usually indicate that the tables are not aligned with the workload. I'd check whether the join columns can be used for compatible distribution or whether pre-aggregation can reduce the amount of data moved. I'd validate the execution plan and bytes moved before and after.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse load into a columnstore table is slow.

My Answer:

I'd inspect the load method, batch size, table distribution, and columnstore state. Large, well-formed batches generally help columnstore compression, while tiny loads can create fragmented rowgroups. I'd also check whether statistics or other downstream maintenance are being performed unnecessarily during every small load. After changing the load pattern, I'd compare load duration and columnstore health.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A query plan estimates far fewer rows than actually exist.

My Answer:

I'd treat that as a statistics or data-distribution problem until proven otherwise. I'd inspect statistics on the columns used in joins and filters and check their last update time. If the data distribution changed materially, I'd update the relevant statistics rather than all statistics blindly. Then I'd rerun the query and compare estimated versus actual row counts.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A team updates statistics on every column after every small load.

My Answer:

I'd make the maintenance more selective. Microsoft guidance emphasizes updating statistics when data changes materially and notes that join, WHERE, and GROUP BY columns are especially valuable. Date columns may need frequent updates when new dates are loaded. I'd monitor stats age and data-change patterns, then update the statistics that actually affect important queries.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A historical correction requires updating millions of rows.

My Answer:

I'd first determine whether the correction affects a large, well-defined slice of the table. For large historical changes, I'd consider a CTAS-based approach where it is appropriate rather than issuing huge numbers of row-by-row modifications. I'd validate the replacement dataset before switching it into the serving path. The key is reducing expensive distributed update work while preserving the target schema and data.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse query is slow because the clustered columnstore is unhealthy.

My Answer:

I'd verify the columnstore condition and identify whether fragmentation or poor rowgroup quality is contributing to the issue. I'd also rule out outdated statistics and data movement because those are common performance causes. If maintenance is justified, I'd perform it in a controlled window and then compare compression, scan behavior, and query performance rather than rebuilding indexes automatically.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A source table has a very high-cardinality ID column.

My Answer:

I'd avoid choosing a distribution key only because the column is unique. A good distribution key needs to spread rows evenly and also support the dominant join patterns. I'd examine frequency distribution, join usage, and data movement before deciding. If the ID is evenly distributed but rarely used in joins, another design may be more useful for the warehouse workload.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A query filters heavily on a date column but scans the whole table.

My Answer:

I'd check the table design and the actual query plan first. If the table is large, I'd evaluate whether the physical organization and statistics allow the engine to eliminate unnecessary data. I'd also make sure the predicate is written in a way that does not force an implicit conversion or prevent efficient filtering. Then I'd compare scanned data and runtime.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A stored procedure behaves differently for different parameter values.

My Answer:

I'd capture the actual execution behavior for representative parameter values and check whether the data distribution changes the optimal plan. I'd also verify statistics and parameter handling. Rather than assuming a single plan is always best, I'd measure the expensive cases and choose a supported mitigation based on the evidence. I'd test both small and large parameter values after the change.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse load from ADLS needs reliable bulk ingestion.

My Answer:

I'd use the supported bulk loading pattern such as COPY and make the column mapping and authentication explicit. I'd validate file format, encoding, row terminators, and target distribution. For production, I'd also record the source batch or file identifiers so a failed load can be safely retried. After the load, I'd reconcile source and target row counts.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A daily incremental load contains late updates to previous dates.

My Answer:

I'd define the change boundary using the source's reliable change timestamp or sequence, not simply today's date. Then I'd process a controlled replay window and update the affected target rows or partitions. I'd make the operation rerunnable and reconcile the corrected date range. I'd also monitor how often late updates arrive because that determines the appropriate replay window.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse table is loaded correctly but reports show duplicate rows.

My Answer:

I'd trace the duplication back through the pipeline rather than deduplicating the final report blindly. I'd compare source keys, staging counts, and target keys at each step. Common causes include repeated batch loads, incorrect joins, or missing business-key constraints. Once the source of duplication is identified, I'd make the load idempotent and add a duplicate-key validation.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A query joins a fact table to the same dimension multiple times.

My Answer:

I'd inspect whether the joins are genuinely needed and whether the dimension attributes can be resolved once in a reusable intermediate result. I'd also project only the required dimension columns. If the repeated joins create large data movement, I'd examine distribution alignment and pre-aggregation. I'd use the execution plan to confirm whether the optimizer already eliminates redundant work before rewriting the query.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A dedicated SQL pool has uneven workload across compute nodes.

My Answer:

I'd check distribution skew and the most expensive queries using the available workload telemetry. If one distribution receives much more data or work, the distribution key may be the cause. I'd also check whether a query is causing a large data movement operation. The fix could be a table redesign, query rewrite, or workload scheduling change depending on the evidence.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A query is slow after a schema change that added several columns.

My Answer:

I'd compare the new query plan and table scan behavior with the previous version. Extra columns can increase row width and I/O even if the query does not logically need them, especially when `SELECT *` is used. I'd replace broad projections with explicit columns and check whether statistics or columnstore health changed. Then I'd measure bytes processed and runtime.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A staging table receives repeated copies of the same batch.

My Answer:

I'd add a batch identifier or source file identifier and make the ingestion process check whether that batch has already been committed. If the target is being replaced, I'd use a controlled staging-and-swap pattern; if it is incremental, I'd use a key-based merge strategy where supported. I'd test retrying the same batch twice and verify that the target remains unchanged after the first successful load.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A warehouse query uses `SELECT *` across several large tables.

My Answer:

I'd replace `SELECT *` with only the columns required by the report. This reduces row width, data movement, memory, and potentially scan work. I'd also inspect joins and filters to see whether the query can reduce data earlier. I'd compare the execution plan and runtime after the rewrite and use the result as a coding standard for similar reports.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A large fact table is frequently filtered by customer_id.

My Answer:

I'd examine whether customer_id is also a common join key and whether its distribution is sufficiently even. If it is a good distribution candidate, hash distribution could reduce data movement for related joins, but I would validate skew first. If it is only a filter and not a major join key, I'd avoid redesigning distribution solely for that predicate.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse query has outdated statistics on a rapidly changing date column.

My Answer:

I'd update the date-related statistics according to the data-change pattern because new dates can materially change the histogram. Microsoft guidance specifically calls out date columns as candidates for frequent updates when new load or transaction dates are added. I'd then rerun the query and confirm whether the estimated cardinality and plan improved.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A report query is slow because of repeated scalar expressions.

My Answer:

I'd check whether the expressions are being recalculated over a large dataset and whether they can be computed once in a staging or derived result. I'd also make sure predicates remain sargable and do not force unnecessary conversions. I'd use the actual execution plan and query metrics to determine whether expression evaluation is truly material before refactoring.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A table needs a new column but historical data is large.

My Answer:

I'd separate the schema change from the historical backfill. First I'd add the column with an appropriate default or nullable behavior, then backfill in controlled batches or a CTAS-based approach if the change is large enough. I'd monitor transaction size and workload impact. Finally, I'd validate both historical and newly arriving records.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A query uses functions on a filtered date column and becomes slow.

My Answer:

I'd inspect whether the function prevents the engine from using the predicate efficiently. If the requirement can be expressed as a range on the raw column, I'd rewrite it that way. For example, instead of transforming every timestamp before filtering, I'd use explicit start and end boundaries. I'd compare the resulting plan and scan behavior.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A pipeline needs to validate source-to-target counts.

My Answer:

I'd capture counts at each major boundary: source extraction, staging, transformation, and final target. Counts alone are not enough for updates, so I'd also reconcile key totals, null counts, and business measures where appropriate. I'd store the validation result with the batch ID and fail or alert according to the business threshold.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A fact table receives millions of rows every hour.

My Answer:

I'd design the ingestion around large, controlled batches rather than many tiny transactions. I'd validate the distribution key, columnstore behavior, and statistics maintenance schedule. I'd also isolate ingestion from reporting workload where possible through scheduling and workload management. The goal is stable throughput while keeping the serving layer queryable.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A report needs near-real-time data but the warehouse load is batch-oriented.

My Answer:

I'd clarify the required freshness because 'real time' can mean seconds, minutes, or an hour. If the requirement is minutes, a frequent incremental pattern may be enough; if it is seconds, I'd consider whether the dedicated SQL pool is the right serving layer. I'd avoid forcing a batch warehouse into an operational workload without measuring latency and concurrency requirements.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse query is slow because of an implicit data-type conversion.

My Answer:

I'd identify the columns involved in the join or filter and make their data types consistent across the query. Implicit conversions can add CPU work and interfere with efficient access patterns. I'd inspect the plan for conversion operators and compare runtime after explicitly casting at a controlled boundary. I'd also fix the underlying schema mismatch where possible.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A dimension table has frequent small updates.

My Answer:

I'd first decide whether the dimension truly needs row-level updates or whether a periodic rebuild is simpler and more efficient. For large historical changes, a CTAS-based rebuild can sometimes be more appropriate. I'd choose based on update volume, table size, downstream dependencies, and maintenance window. I'd validate surrogate keys and history rules before changing the load pattern.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A query joins two huge tables and runs out of time.

My Answer:

I'd inspect data movement, distribution alignment, filtering, and statistics before increasing the timeout. If both tables can be filtered substantially before the join, I'd push those filters earlier. If the join keys align with distribution, movement can be reduced. I'd also consider pre-aggregating one side if the report only needs summarized data. Then I'd validate the plan and elapsed time.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A production load fails halfway through.

My Answer:

I'd determine whether the load is transactional at the required scope or whether it left partial data in the target. I'd use batch IDs and staging tables so the load can be retried without duplicating successful rows. Before rerunning, I'd reconcile what reached the target. I'd then improve the pipeline so each batch has a clear committed or failed state.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A team wants to create statistics on every column in a very large table.

My Answer:

I'd start with the columns that materially affect query plans: join keys, filter columns, and grouping columns. Creating and maintaining statistics on every column can add unnecessary work. I'd use workload evidence and statistics metadata to prioritize. If query patterns change, I'd revisit the statistics set rather than keeping an oversized maintenance process forever.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A query is fast in development but slow in production.

My Answer:

I'd compare data volume, distribution, statistics freshness, concurrency, resource allocation, and the actual execution plan. Development often has a much smaller dataset, so skew and data movement may not appear. I'd reproduce the production query shape and inspect telemetry. I'd then fix the specific production bottleneck rather than copying development settings or adding arbitrary indexes.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A large table is frequently rebuilt from a source snapshot.

My Answer:

I'd consider whether the full rebuild is necessary or whether the source can provide incremental changes. If a rebuild is required, I'd use a controlled CTAS pattern where appropriate, validate the new table, and then switch consumers safely. I'd avoid exposing a partially rebuilt table. I'd also compare the operational cost of the rebuild with the business value of snapshot-based processing.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A report is timing out during peak hours.

My Answer:

I'd separate query complexity from concurrency pressure. I'd inspect workload telemetry to see whether the query itself is slow or waiting behind other work. Then I'd optimize the query plan, reduce unnecessary scans, and consider scheduling or workload management changes if concurrency is the main issue. I'd validate the same report under realistic peak conditions.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A pipeline needs to delete a large historical slice.

My Answer:

I'd avoid blindly issuing a massive row-by-row delete. I'd first determine whether the affected data can be replaced or rebuilt more efficiently, potentially using CTAS for a large historical transformation. I'd validate the retained dataset before switching it into service. If a direct delete is required, I'd plan the operation around transaction size and workload impact.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse query uses a CTE many times and is unexpectedly slow.

My Answer:

I'd inspect the actual execution plan rather than assuming the CTE is materialized. If the same expensive logic is repeated, I'd consider materializing an intermediate result when that improves the workload. I'd also reduce columns and rows before the repeated logic. The decision should be based on measured runtime and resource usage, not on the presence of a CTE alone.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A table has a poor distribution key chosen years ago.

My Answer:

I'd use current workload evidence to decide whether redesign is justified. I'd measure skew, data movement, high-cost joins, and the size of the table. If the distribution is clearly hurting major workloads, I'd build and validate a new table with a better strategy, then migrate consumers in a controlled way. I'd avoid changing a stable table just because a different key looks theoretically better.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A source sends a correction for a closed accounting period.

My Answer:

I'd treat it as a controlled restatement rather than a normal daily append. I'd identify the affected business keys and accounting period, process the correction idempotently, and reconcile financial totals before and after. I'd also record the correction batch and reason for auditability. The key is preserving an explainable history while ensuring current reporting reflects the approved correction.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A warehouse needs to expose a stable semantic layer to BI users.

My Answer:

I'd keep raw and staging structures separate from business-facing views or tables. The semantic layer should expose stable names, business definitions, and controlled joins so BI users do not depend on ingestion internals. I'd document grain and key definitions, then validate common reports against known business totals. This reduces the chance that pipeline changes unexpectedly break reporting.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse load is successful but row counts do not reconcile.

My Answer:

I'd compare counts by source file, batch ID, and transformation step rather than looking only at the final total. I'd check rejected rows, duplicate handling, filters, and joins for row multiplication or loss. I'd preserve a load audit record with source and target counts. Once the mismatch is isolated, I'd fix the specific stage and add a regression check.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A production SQL change needs to be deployed safely.

My Answer:

I'd separate the schema change, data migration, and application cutover so each step can be validated. I'd test the change against representative data, capture the rollback plan, and monitor the first production run closely. For large changes, I'd use a staged approach rather than one long blocking operation. I'd document exactly how to verify success before removing the old path.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A team wants to optimize a query without changing the table design.

My Answer:

I'd start with query-level changes: remove unnecessary columns, push filters early, simplify joins, eliminate repeated calculations, and make data types compatible. I'd inspect the actual plan after each change so I know which modification mattered. If the query remains expensive, I'd explain which limitation comes from the physical table design rather than promising that SQL syntax alone can solve it.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse workload needs an operational monitoring checklist.

My Answer:

I'd monitor query duration, failed requests, data movement, statistics freshness, columnstore health, distribution skew, load throughput, and reconciliation metrics. I'd set thresholds around business impact rather than alerting on every small fluctuation. For incidents, I'd capture the request ID and plan details so the team can move from alert to diagnosis quickly. I'd review the checklist as the workload evolves.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse table has an uneven distribution after a redesign.

My Answer:

I'd compare row counts across distributions and identify which key values are responsible for the imbalance. I'd then evaluate whether another distribution key or strategy better matches the workload. I'd test the proposed design using representative data and major joins before migrating the production table.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A dedicated SQL pool load creates many small batches.

My Answer:

I'd consolidate the load into larger controlled batches where the source pattern allows it. Very small loads can increase overhead and can be less favorable for columnstore storage. I'd compare load throughput, rowgroup quality, and downstream query performance after changing the batch pattern.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse report needs a repeatable reconciliation process.

My Answer:

I'd create a batch-level audit record containing source counts, target counts, key totals, rejected records, and processing timestamps. The reconciliation should run automatically after each load and compare against defined tolerances. If a critical metric fails, the pipeline should alert or stop publication rather than silently producing a questionable report.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A warehouse migration changes data types between source and target.

My Answer:

I'd map each source type explicitly and test boundary values before the migration. I'd pay particular attention to numeric precision, string length, timestamps, and nullability. I'd validate sample records and aggregate totals before switching consumers. If an implicit conversion is required, I'd make it explicit at a controlled transformation boundary.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

A Synapse workload needs a safe production cutover.

My Answer:

I'd build the new table or pipeline alongside the existing path, validate counts and business metrics, and keep the old path available until acceptance checks pass. I'd then switch consumers in a controlled step and monitor the first production runs. I'd document the rollback point before starting the cutover so recovery does not depend on improvisation.

Follow-up Question:

What execution-plan or workload evidence would you check to prove the change improved the warehouse?

Follow-up Answer:

I'd compare elapsed time, rows processed, data movement, estimated versus actual rows, and the relevant workload metrics before and after. I'd also verify that the result set is unchanged.

Interview Question:

Write a query to find departments with more than 10 employees.

My Answer:

I'd aggregate employees by department and filter the aggregate with HAVING.

```
SELECT department_id, COUNT(*) AS employee_count
FROM Employee
GROUP BY department_id
HAVING COUNT(*) > 10;
```

Follow-up Question:

Why HAVING?

Follow-up Answer:

HAVING filters aggregate results, while WHERE filters rows before aggregation.

Interview Question:

Write a query to find customers with no orders.

My Answer:

I'd use NOT EXISTS because the requirement is an existence check.

```
SELECT c.customer_id
FROM Customer c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders o WHERE o.customer_id = c.customer_id
);
```

Follow-up Question:

Why not NOT IN?

Follow-up Answer:

NOT EXISTS avoids several NULL-related surprises that can occur with NOT IN.

Interview Question:

Write a query to find the latest order per customer.

My Answer:

I'd rank orders within each customer and keep row number one.

```
WITH x AS (  
  SELECT o.*, ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date DESC, order_id  
  DESC) rn  
  FROM Orders o  
)  
SELECT * FROM x WHERE rn = 1;
```

Follow-up Question:

Why use order_id?

Follow-up Answer:

It provides a deterministic tie-breaker when order dates match.

Interview Question:

Write a query to find duplicate emails.

My Answer:

I'd group by email and use HAVING COUNT greater than one.

```
SELECT email, COUNT(*) AS cnt
FROM Customer
GROUP BY email
HAVING COUNT(*) > 1;
```

Follow-up Question:

Would you delete them directly?

Follow-up Answer:

No. I'd first define the authoritative record and preview the cleanup.

Interview Question:

Write a query to calculate running sales by customer.

My Answer:

I'd use a window SUM ordered by the transaction date.

```
SELECT customer_id, order_date, amount,  
       SUM(amount) OVER(PARTITION BY customer_id ORDER BY order_date, order_id ROWS  
       UNBOUNDED PRECEDING) AS running_sales  
FROM Orders;
```

Follow-up Question:

Why include order_id?

Follow-up Answer:

It makes same-date ordering deterministic.

Interview Question:

Write a query to find employees above the company average salary.

My Answer:

I'd calculate the overall average and compare each employee against it.

```
SELECT employee_id, salary
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Follow-up Question:

What about NULL salary?

Follow-up Answer:

NULL does not satisfy the comparison, which is usually appropriate unless the business rule says otherwise.

Interview Question:

Write a query to return the top 3 products by revenue.

My Answer:

I'd aggregate revenue by product and rank the totals.

```
SELECT TOP (3) product_id, SUM(amount) AS revenue
FROM Sales
GROUP BY product_id
ORDER BY revenue DESC;
```

Follow-up Question:

What if ties matter?

Follow-up Answer:

I'd use `DENSE_RANK` instead of `TOP` when all tied products should be returned.

Interview Question:

Write a query to find employees hired in the last 30 days.

My Answer:

I'd use a parameterized reference date and a date range.

```
SELECT employee_id, hire_date
FROM Employee
WHERE hire_date >= DATEADD(day, -30, @ReferenceDate)
AND hire_date < DATEADD(day, 1, @ReferenceDate);
```

Follow-up Question:

Why parameterize the date?

Follow-up Answer:

It makes testing and reporting reproducible.

Interview Question:

Write a query to calculate order count by month.

My Answer:

I'd group orders at month grain using an explicit date expression.

```
SELECT YEAR(order_date) AS order_year, MONTH(order_date) AS order_month, COUNT(*) AS  
order_count  
FROM Orders  
GROUP BY YEAR(order_date), MONTH(order_date);
```

Follow-up Question:

What timezone issue can matter?

Follow-up Answer:

If timestamps are UTC, the business reporting timezone should be applied before deriving the date.

Interview Question:

Write a query to find the highest salary in each department.

My Answer:

I'd aggregate salary by department and take MAX.

```
SELECT department_id, MAX(salary) AS max_salary
FROM Employee
GROUP BY department_id;
```

Follow-up Question:

How do you return the employee too?

Follow-up Answer:

Use ROW_NUMBER or DENSE_RANK and filter the top-ranked row or rows.

Interview Question:

Write a query to find customers whose total spend exceeds 10000.

My Answer:

I'd aggregate orders by customer and use HAVING.

```
SELECT customer_id, SUM(amount) AS total_spend
FROM Orders
GROUP BY customer_id
HAVING SUM(amount) > 10000;
```

Follow-up Question:

Why not WHERE SUM(amount)?

Follow-up Answer:

Aggregate expressions are filtered with HAVING.

Interview Question:

Write a query to return the second order for each customer.

My Answer:

I'd rank orders within customer and filter rank two.

```
WITH x AS (  
  SELECT o.*, ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date, order_id) rn  
  FROM Orders o  
)  
SELECT * FROM x WHERE rn = 2;
```

Follow-up Question:

What if a customer has one order?

Follow-up Answer:

That customer simply has no second-order row.

Interview Question:

Write a query to find customers who ordered in January but not February.

My Answer:

I'd use EXISTS for January and NOT EXISTS for February.

```
SELECT c.customer_id
FROM Customer c
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id=c.customer_id AND o.order_date >=
'2026-01-01' AND o.order_date < '2026-02-01')
AND NOT EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id=c.customer_id AND o.order_date >=
'2026-02-01' AND o.order_date < '2026-03-01');
```

Follow-up Question:

Why use half-open ranges?

Follow-up Answer:

They avoid time-of-day boundary problems.

Interview Question:

Write a query to find employees who earn more than their manager.

My Answer:

I'd self-join Employee to itself using manager_id.

```
SELECT e.employee_id  
FROM Employee e  
JOIN Employee m ON e.manager_id = m.employee_id  
WHERE e.salary > m.salary;
```

Follow-up Question:

Why self-join?

Follow-up Answer:

Employee and manager rows are stored in the same table.

Interview Question:

Write a query to find the first order date per customer.

My Answer:

I'd use MIN at customer grain.

```
SELECT customer_id, MIN(order_date) AS first_order_date
FROM Orders
GROUP BY customer_id;
```

Follow-up Question:

When would you use ROW_NUMBER instead?

Follow-up Answer:

When you need columns from the complete first-order row.

Interview Question:

Write a query to calculate average order value.

My Answer:

I'd calculate total order amount divided by order count or use AVG at order grain.

```
SELECT AVG(amount) AS avg_order_value  
FROM Orders;
```

Follow-up Question:

What should be excluded?

Follow-up Answer:

Cancelled or test orders should be excluded if the business definition does not treat them as revenue-bearing orders.

Interview Question:

Write a query to find products never sold.

My Answer:

I'd start from Product and use NOT EXISTS.

```
SELECT p.product_id
FROM Product p
WHERE NOT EXISTS (SELECT 1 FROM Sales s WHERE s.product_id=p.product_id);
```

Follow-up Question:

Why start from Product?

Follow-up Answer:

Never-sold products have no Sales rows.

Interview Question:

Write a query to find orders larger than the customer's average.

My Answer:

I'd use a window average over customer_id.

```
SELECT *
FROM (
  SELECT o.*, AVG(amount) OVER(PARTITION BY customer_id) avg_amount
  FROM Orders o
) x
WHERE amount > avg_amount;
```

Follow-up Question:

What does the window preserve?

Follow-up Answer:

It keeps every order row while adding the comparison value.

Interview Question:

Write a query to calculate year-over-year growth by product.

My Answer:

I'd aggregate by product/year and use LAG.

```
WITH y AS (  
  SELECT product_id, YEAR(order_date) yr, SUM(amount) revenue  
  FROM Sales GROUP BY product_id, YEAR(order_date)  
)  
SELECT *, (revenue - LAG(revenue) OVER(PARTITION BY product_id ORDER BY yr)) /  
  NULLIF(LAG(revenue) OVER(PARTITION BY product_id ORDER BY yr),0) AS yoy_growth  
FROM y;
```

Follow-up Question:

Why NULLIF?

Follow-up Answer:

It prevents division by zero when prior-year revenue is zero.

Interview Question:

Write a query to find consecutive purchase days.

My Answer:

I'd first reduce to distinct customer/date rows, then compare each date with LAG.

```
WITH d AS (SELECT DISTINCT customer_id, CAST(order_date AS date) order_day FROM Orders), x  
AS (SELECT *, LAG(order_day) OVER(PARTITION BY customer_id ORDER BY order_day) prev_day FROM  
d)  
SELECT * FROM x WHERE DATEDIFF(day, prev_day, order_day)=1;
```

Follow-up Question:

Why distinct dates?

Follow-up Answer:

Multiple orders on the same day should count as one active day.

Interview Question:

Write a query to find the top customer in each region.

My Answer:

I'd aggregate revenue by region/customer and rank within region.

```
WITH x AS (  
  SELECT region, customer_id, SUM(amount) revenue,  
         DENSE_RANK() OVER(PARTITION BY region ORDER BY SUM(amount) DESC) rnk  
  FROM Sales GROUP BY region, customer_id  
)  
SELECT * FROM x WHERE rnk=1;
```

Follow-up Question:

Why DENSE_RANK?

Follow-up Answer:

It returns all customers tied for the highest regional revenue.

Interview Question:

Write a query to find orders delivered late.

My Answer:

I'd compare actual delivery date with promised date and explicitly exclude invalid statuses.

```
SELECT order_id
FROM Orders
WHERE actual_delivery_date > promised_delivery_date;
```

Follow-up Question:

What about NULL delivery dates?

Follow-up Answer:

They should be handled by an explicit business rule rather than automatically classified as late.

Interview Question:

Write a query to calculate monthly revenue.

My Answer:

I'd aggregate revenue by year and month.

```
SELECT YEAR(order_date) yr, MONTH(order_date) mn, SUM(amount) revenue
FROM Sales
GROUP BY YEAR(order_date), MONTH(order_date);
```

Follow-up Question:

What should be defined?

Follow-up Answer:

The reporting timezone and whether refunds are included.

Interview Question:

Write a query to find the third-highest salary.

My Answer:

I'd use DENSE_RANK for the third distinct salary.

```
WITH x AS (SELECT *, DENSE_RANK() OVER(ORDER BY salary DESC) rnk FROM Employee)  
SELECT * FROM x WHERE rnk=3;
```

Follow-up Question:

What if exactly one row is required?

Follow-up Answer:

Use ROW_NUMBER with a deterministic tie-breaker.

Interview Question:

Write a query to calculate customer lifetime revenue.

My Answer:

I'd aggregate qualifying revenue by customer.

```
SELECT customer_id, SUM(amount) lifetime_revenue
FROM Orders
GROUP BY customer_id;
```

Follow-up Question:

What business rules matter?

Follow-up Answer:

Refunds, cancellations, currency conversion, and test orders should be explicitly defined.

Interview Question:

Write a query to find departments with no employees.

My Answer:

I'd use a LEFT JOIN from departments to employees and filter the missing side.

```
SELECT d.department_id
FROM Department d
LEFT JOIN Employee e ON e.department_id=d.department_id
WHERE e.employee_id IS NULL;
```

Follow-up Question:

Why not start from Employee?

Follow-up Answer:

An empty department has no employee row to drive the result.

Interview Question:

Write a query to find the maximum order per customer.

My Answer:

I'd rank orders by amount within customer and select rank one.

```
WITH x AS (SELECT o.*, ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY amount DESC,
order_id DESC) rn FROM Orders o)
SELECT * FROM x WHERE rn=1;
```

Follow-up Question:

Why not MAX(amount)?

Follow-up Answer:

MAX returns the value but not the complete order row.

Interview Question:

Write a query to calculate cancellation rate.

My Answer:

I'd use conditional aggregation over the eligible order population.

```
SELECT 100.0 * SUM(CASE WHEN status='Cancelled' THEN 1 ELSE 0 END) / NULLIF(COUNT(*),0)
cancellation_pct
FROM Orders;
```

Follow-up Question:

What should be excluded?

Follow-up Answer:

Test orders or other non-business orders should be excluded if the metric definition requires it.

Interview Question:

Write a query to find customers with exactly one order.

My Answer:

I'd group by customer and use `HAVING COUNT(*)=1`.

```
SELECT customer_id
FROM Orders
GROUP BY customer_id
HAVING COUNT(*)=1;
```

Follow-up Question:

What about cancelled orders?

Follow-up Answer:

Apply the business-status filter before grouping.

Interview Question:

Write a query to find the latest status for every order.

My Answer:

I'd rank status-history records by timestamp descending.

```
WITH x AS (SELECT s.*, ROW_NUMBER() OVER(PARTITION BY order_id ORDER BY status_time DESC,  
status_event_id DESC) rn FROM OrderStatus s)  
SELECT * FROM x WHERE rn=1;
```

Follow-up Question:

Why status_event_id?

Follow-up Answer:

It makes equal timestamps deterministic.

Interview Question:

Write a query to find customers whose latest order is older than 90 days.

My Answer:

I'd calculate MAX(order_date) per customer and compare with a reference date.

```
SELECT customer_id, MAX(order_date) latest_order
FROM Orders
GROUP BY customer_id
HAVING MAX(order_date) < DATEADD(day,-90,@ReferenceDate);
```

Follow-up Question:

Why parameterize the reference date?

Follow-up Answer:

It makes the result repeatable in tests and scheduled reports.

Interview Question:

Write a query to calculate product share of category revenue.

My Answer:

I'd aggregate product revenue and divide by category total with a window SUM.

```
WITH p AS (SELECT category_id, product_id, SUM(amount) revenue FROM Sales GROUP BY
category_id, product_id)
SELECT *, revenue / NULLIF(SUM(revenue) OVER(PARTITION BY category_id),0) category_share
FROM p;
```

Follow-up Question:

Why aggregate first?

Follow-up Answer:

The denominator is total category revenue.

Interview Question:

Write a query to find the first purchase after registration.

My Answer:

I'd join orders to customers and take the minimum qualifying order date.

```
SELECT c.customer_id, MIN(o.order_date) first_purchase
FROM Customer c JOIN Orders o ON o.customer_id=c.customer_id AND
o.order_date>=c.registration_date
GROUP BY c.customer_id;
```

Follow-up Question:

What if there is no purchase?

Follow-up Answer:

A LEFT JOIN can preserve those customers with NULL first_purchase.

Interview Question:

Write a query to compare current order value with the previous order.

My Answer:

I'd use LAG by customer ordered by date and order ID.

```
SELECT o.*, LAG(amount) OVER(PARTITION BY customer_id ORDER BY order_date, order_id)
previous_amount
FROM Orders o;
```

Follow-up Question:

How do you identify increases?

Follow-up Answer:

Compare amount with previous_amount and exclude the first row per customer.

Interview Question:

Write a query to find products with no sales in the last 90 days.

My Answer:

I'd start from products and use NOT EXISTS with a date filter.

```
SELECT p.product_id
FROM Product p
WHERE NOT EXISTS (SELECT 1 FROM Sales s WHERE s.product_id=p.product_id AND
s.sale_date>=DATEADD(day,-90,@ReferenceDate));
```

Follow-up Question:

Why start from Product?

Follow-up Answer:

Products with no recent sales still need to appear.

Interview Question:

Write a query to find the average number of products per order.

My Answer:

I'd first count products per order and then average those order-level counts.

```
WITH x AS (SELECT order_id, COUNT(DISTINCT product_id) product_count FROM OrderLine GROUP BY  
order_id)  
SELECT AVG(CAST(product_count AS decimal(18,4))) avg_products_per_order FROM x;
```

Follow-up Question:

Why two levels?

Follow-up Answer:

The metric is an average per order, so each order must contribute one value.

Interview Question:

Write a query to find customers who ordered in every month of a year.

My Answer:

I'd count distinct order months per customer and compare with twelve for a complete year.

```
SELECT customer_id
FROM Orders
WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01'
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date))=12;
```

Follow-up Question:

What if the reporting period is incomplete?

Follow-up Answer:

Parameterize the expected month count instead of always requiring twelve.

Interview Question:

Write a query to find the highest revenue day per month.

My Answer:

I'd aggregate daily revenue, rank days within each month, and keep rank one.

```
WITH d AS (SELECT CAST(order_date AS date) order_day, SUM(amount) revenue FROM Sales GROUP BY CAST(order_date AS date)), x AS (SELECT *, DENSE_RANK() OVER(PARTITION BY YEAR(order_day), MONTH(order_day) ORDER BY revenue DESC) rnk FROM d) SELECT * FROM x WHERE rnk=1;
```

Follow-up Question:

Why aggregate by day first?

Follow-up Answer:

The requirement is to compare daily totals.

Interview Question:

Write a query to find categories above average category revenue.

My Answer:

I'd aggregate category revenue and compare each category with the average of those category totals.

```
WITH c AS (SELECT category_id,SUM(amount) revenue FROM Sales GROUP BY category_id)
SELECT * FROM c WHERE revenue>(SELECT AVG(revenue) FROM c);
```

Follow-up Question:

Why use a CTE?

Follow-up Answer:

It makes the two aggregation levels explicit and reusable.

Interview Question:

Write a query to find customers with three consecutive purchase months.

My Answer:

I'd reduce the data to one row per customer and month, use LAG to detect sequence breaks, then group each uninterrupted sequence.

```
WITH m AS (  
  SELECT DISTINCT customer_id, DATEFROMPARTS(YEAR(order_date),MONTH(order_date),1) AS  
    month_start  
  FROM Orders  
) , x AS (  
  SELECT *, LAG(month_start) OVER(PARTITION BY customer_id ORDER BY month_start) AS  
    prev_month  
  FROM m  
) , g AS (  
  SELECT *, SUM(CASE WHEN prev_month IS NULL OR DATEDIFF(month,prev_month,month_start)<>1  
    THEN 1 ELSE 0 END)  
    OVER(PARTITION BY customer_id ORDER BY month_start ROWS UNBOUNDED PRECEDING) AS grp  
  FROM x  
)  
SELECT customer_id FROM g GROUP BY customer_id,grp HAVING COUNT(*)>=3;
```

Follow-up Question:

What must be clarified about three consecutive months?

Follow-up Answer:

Whether it means at least three or exactly three months, and which reporting calendar and timezone define a month.

Interview Question:

Write a query to calculate cumulative distinct customers by month.

My Answer:

I'd identify each customer's first purchase month and count customers whose first month is on or before each reporting month.

```
WITH first_purchase AS (  
    SELECT customer_id, DATEFROMPARTS(YEAR(MIN(order_date)),MONTH(MIN(order_date)),1) AS  
    first_month  
    FROM Orders GROUP BY customer_id  
)  
SELECT c.month_start, COUNT(f.customer_id) AS cumulative_customers  
FROM Calendar c LEFT JOIN first_purchase f ON f.first_month<=c.month_start  
GROUP BY c.month_start;
```

Follow-up Question:

Why use first purchase month?

Follow-up Answer:

Each customer enters the cumulative population once, so the first purchase month defines when that customer enters the metric.

Interview Question:

Write a query to find orders containing more than five distinct products.

My Answer:

I'd aggregate order lines by order and filter with HAVING.

```
SELECT order_id
FROM OrderLine
GROUP BY order_id
HAVING COUNT(DISTINCT product_id)>5;
```

Follow-up Question:

What is the grain?

Follow-up Answer:

One row per order after aggregation.

Interview Question:

Write a query to find revenue from the top 10 customers.

My Answer:

I'd aggregate customer revenue, select the top ten, and divide their total by overall revenue.

```
WITH c AS (SELECT customer_id, SUM(amount) revenue FROM Sales GROUP BY customer_id), t AS  
(SELECT TOP (10) * FROM c ORDER BY revenue DESC) SELECT 100.0 * SUM(revenue) / (SELECT  
SUM(revenue) FROM c) top10_pct FROM t;
```

Follow-up Question:

How do ties affect it?

Follow-up Answer:

TOP returns exactly ten; a rank-based rule can include more customers if ties are required.

Interview Question:

Write a query to find the first month a customer crossed a revenue threshold.

My Answer:

I'd aggregate monthly customer revenue, filter months above the threshold, and select the earliest qualifying month.

```
WITH m AS (SELECT customer_id,DATEFROMPARTS(YEAR(order_date),MONTH(order_date),1)
month_start,SUM(amount) revenue FROM Orders GROUP BY
customer_id,DATEFROMPARTS(YEAR(order_date),MONTH(order_date),1)) SELECT
customer_id,MIN(month_start) first_month FROM m WHERE revenue>10000 GROUP BY customer_id;
```

Follow-up Question:

Why filter before MIN?

Follow-up Answer:

Only qualifying months should participate in the first-occurrence calculation.

Interview Question:

Write a query to find the top 10 percent of customers by revenue.

My Answer:

I'd aggregate customer revenue and apply the ranking rule required by the business definition. For exact population size I'd use ROW_NUMBER; for tie-inclusive results I'd use a rank function.

```
WITH c AS (SELECT customer_id, SUM(amount) revenue FROM Sales GROUP BY customer_id), x AS  
(SELECT *, ROW_NUMBER() OVER(ORDER BY revenue DESC) rn, COUNT(*) OVER() total_count FROM c)  
SELECT * FROM x WHERE rn <= CEILING(total_count * 0.10);
```

Follow-up Question:

What should be clarified?

Follow-up Answer:

Whether ties can expand the top population.

Interview Question:

Write a query to find revenue by weekday.

My Answer:

I'd derive the business date and group by the agreed weekday convention.

```
SELECT DATENAME(weekday,order_date) weekday_name,SUM(amount) revenue FROM Sales GROUP BY  
DATENAME(weekday,order_date);
```

Follow-up Question:

What is the timezone concern?

Follow-up Answer:

UTC timestamps may fall on a different business day after local conversion.

Interview Question:

Write a query to find the month with highest revenue per year.

My Answer:

I'd aggregate monthly revenue, rank months within year, and select rank one.

```
WITH m AS (SELECT YEAR(order_date) yr, MONTH(order_date) mn, SUM(amount) revenue FROM Sales  
GROUP BY YEAR(order_date), MONTH(order_date)), x AS (SELECT *, DENSE_RANK() OVER(PARTITION BY  
yr ORDER BY revenue DESC) rnk FROM m) SELECT * FROM x WHERE rnk=1;
```

Follow-up Question:

What if two months tie?

Follow-up Answer:

DENSE_RANK returns both tied months.

Interview Question:

Write a query to find customers whose latest order is also their highest-value order.

My Answer:

I'd rank each customer's orders by recency and calculate the customer maximum in the same pass.

```
WITH x AS (  
  SELECT o.*, ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date DESC,order_id  
  DESC) AS rn,  
  MAX(amount) OVER(PARTITION BY customer_id) AS max_amount  
  FROM Orders o  
)  
SELECT customer_id,order_id,order_date,amount FROM x WHERE rn=1 AND amount=max_amount;
```

Follow-up Question:

What edge case matters here?

Follow-up Answer:

If several orders share the maximum amount, the latest-order rule still needs a deterministic tie-breaker such as order_id.

Interview Question:

Write a query to find early deliveries.

My Answer:

I'd compare actual delivery date with promised delivery date after excluding cancelled or invalid orders.

```
SELECT order_id FROM Orders WHERE actual_delivery_date<promised_delivery_date;
```

Follow-up Question:

What should be clarified?

Follow-up Answer:

Whether same-day delivery counts as early and how missing dates are treated.

Interview Question:

Write a query to calculate average revenue per active customer by month.

My Answer:

I'd aggregate monthly revenue and distinct active customers at the same month grain, then divide using decimal arithmetic.

```
SELECT YEAR(order_date) yr, MONTH(order_date) mn, SUM(amount)/NULLIF(COUNT(DISTINCT customer_id),0) revenue_per_customer FROM Sales GROUP BY YEAR(order_date), MONTH(order_date);
```

Follow-up Question:

What is the denominator?

Follow-up Answer:

Distinct customers with qualifying activity in that month.